

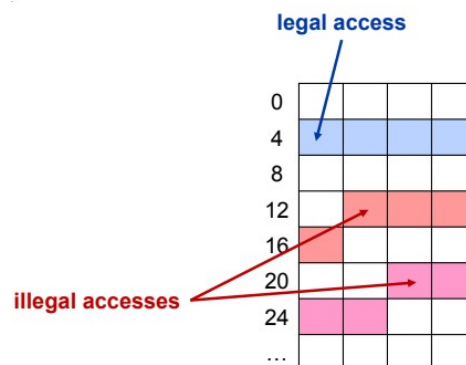
MIPS Memory Instructions and Addressing

Memory Instructions

- Memory is used for storing composite data: arrays, structures, dynamic data.
- Arithmetic operations cannot be performed directly on memory values.
- Separate instructions are used to:
 - Load values from memory into registers (**lw**, **lb**, etc.)
 - Store results from registers to memory (**sw**, **sb**, etc.)
- Unlike some architectures, MIPS does not allow operations directly on memory operands.

Memory Addressing in MIPS

- Memory is **byte-addressed**. Each address points to an 8-bit byte.
- A **word** in MIPS = 4 bytes = 32 bits.
- Word addresses are **aligned**, i.e., must be multiples of 4.
- Accessing non-aligned word addresses is illegal.



Endianness

- **Big Endian (MIPS):** Most significant byte (MSB) stored at lowest address.
- **Little Endian (x86):** Least significant byte (LSB) stored at lowest address.

Address	0	1	2	3
Big Endian	FF	CC	67	89
Little Endian	89	67	CC	FF

Load/Store Instructions

- `lw $t0, 8($s0)`: Load word from address $\$s0 + 8$ into $\$t0$
- `sw $s1, 12($t1)`: Store word in $\$s1$ into memory at $\$t1 + 12$
- Offset must be a multiple of 4 (for words)
- Offset is limited to 16 bits

Example: Simple Array Access

C Code:

```
A[5] = h + A[3];
```

Assumptions: $\$s2$ contains h , $\$s3$ contains base address of A .

MIPS Code:

```
lw $t0, 12($s3)  # t0 = A[3] (3 * 4 = 12)
add $t0, $s2, $t0
sw $t0, 20($s3)  # A[5] = t0 (5 * 4 = 20)
```

Example: Indexed Access with Variables

C Code:

```
A[i] = h + A[j];
```

Assumptions: $\$s0 = i$, $\$s1 = j$, $\$s2 = h$, $\$s3 = \text{base of } A$

MIPS Code:

```
sll $t0, $s1, 2      # t0 = j * 4
add $t0, $s3, $t0     # t0 = A + j*4
lw  $t0, 0($t0)       # t0 = A[j]
add $t0, $s2, $t0     # t0 = h + A[j]
sll $t1, $s0, 2       # t1 = i * 4
add $t1, $s3, $t1     # t1 = A + i*4
sw  $t0, 0($t1)       # A[i] = t0
```

Registers vs Memory

- Registers are faster than memory.
- Use registers for frequently accessed variables.
- Use memory for less frequent or larger data structures.

Handling Small Data (Bytes and Halfwords)

- Characters (ASCII) require 1 byte (8 bits)
- Unicode (UTF-16) uses 2 bytes (16 bits)
- Use byte/halfword load/store instructions:
 - `lb, lh` : sign-extend to 32-bit
 - `lbu, lhu` : zero-extend to 32-bit
 - `sb, sh` : store byte/halfword

Example: Byte Access

C Code:

```
x[2] = x[0] + x[1];
```

Assumptions: `$s0` contains base address of array `x` (unsigned char)

MIPS Code:

```
lbu $t0, 0($s0)    # Load x[0]
lbu $t1, 1($s0)    # Load x[1]
add $t0, $t0, $t1
sb  $t0, 2($s0)    # Store result to x[2]
```